

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#05)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: Manahil Shahzad

Roll No: 24I-6563

Section: CE-A

Date performed: 15TH September, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1: Indicate which of the assembly instructions listed in Table 1 and Table 2 above are actual instructions and which are pseudo-instructions? How do you know?

Description:
In this task, we identified which control instructions are real hardware instructions and which are pseudo-instructions. Pseudo-instructions are simple shortcuts for the programmer that expand into real instructions during compilation. We simulated these in the Venus simulator and correctly interpreted their classifications.
Code:
Actual Instructions: beq, blt, bge, bne, jal, and jalr. Pseudo-Instructions: ble (expands to bge t1, t0, target), bgt (expands to blt t1, t0, target), and j (expands to jal x0, target).
Output:
beq = Actual Instruction blt = Actual Instruction ble = Pseudo-Instruction bgt = Pseudo-Instruction bge = Actual Instruction bne = Actual Instruction j = Pseudo-Instruction jal = Actual Instruction jalr = Actual Instruction

Task 2: Write a code snippet with just two instructions beq and j (conditional branch and jump). Simulate to understand the difference between them. Explain.

Description:

In this task, we compared how conditional branching and unconditional jumping work using beq and j. The beq instruction only jumps if a specific condition is met, while j always jumps no matter what. We executed and simulated this in the Venus simulator and correctly interpreted the execution flow.

Code:

```
addi s0, x0, 5

addi s1, x0, 5

beq s0, s1, target

addi s2, x0, 10

target:

j end

addi s2, x0, 20

end:
```

Output:

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x00500413	addi x8 x0 5	addi s0, x0, 5	t2 (x7)	0
0x4	0x00500493	addi x9 x0 5	addi s1, x0, 5	s0 (x8)	5
0x8	0x00940463	beq x8 x9 8	beq s0, s1, target	s1 (x9)	0

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x00500413	addi x8 x0 5	addi s0, x0, 5	t2 (x7)	0
0x4	0x00500493	addi x9 x0 5	addi s1, x0, 5	s0 (x8)	5
0x8	0x00940463	beq x8 x9 8	beq s0, s1, target	s1 (x9)	5

0xc	0x00A00913	addi x18 x0 10	addi s2, x0, 10	t2 (x7)	0
0x10	0x0080006F	jal x0 8	j end	s0 (x8)	5
0x14	0x01400913	addi x18 x0 20	addi s2, x0, 20	s1 (x9)	5

Task 3: Run the codes given below, comment each code line by line, and explain what it does:

Description:
In this task, we ran two short programs to see how branch decisions change which lines of code get executed. The first program ran the addition path because the numbers were different, while the second jumped to the subtraction path because the numbers matched. We executed and simulated both in the Venus simulator and correctly interpreted the final register values.
Code:
Code 1 with explanations: <pre>addi s0, s0, 4 #Add 4 to s0 (s0 = 4) addi s1, s1, 6 #Add 6 to s1 (s1 = 6) beq s0, s1, Fin # Check if s0 == s1 (False: 4 != 6, moves to next line) True: add s2, s1, s0 # Add s0 and s1 (s2 = 10) j end # Jump to end label Fin: sub s2, s1, s0 # Subtract s0 from s1 (Skipped) end:</pre> Code 2 with explanations: <pre>addi s0, s0, 4 # Add 4 to s0 (s0 = 4) addi s1, s1, 4 #Add 4 to s1 (s1 = 4) beq s0, s1, Fin #Check if s0 == s1 (True: 4 == 4, branches to Fin) True: add s2, s1, s0 # Add s0 and s1 (Skipped) j end #Jump to end (Skipped) Fin: sub s2, s1, s0 #Subtract s0 from s1 (s2 = 4 - 4 = 0) End:</pre>
Output:
Code 1 Final Registers: s0 = 4, s1 = 6, s2 = 10. Code 2 Final Registers: s0 = 4, s1 = 4, s2 = 0.

Task 4: Convert given C code into RISC-V assembly and simulate using Venus simulator. Consider a=5, b=10, d=4, e=3.

Description:

In this task, we implemented conditional branching using the beq instruction to compare two registers. Since the values were not equal, the program skipped the subtraction block, performed an addition, and jumped to the end. We executed and simulated this code in the Venus simulator and correctly interpreted the final register values and control path.

Code:

```
addi s0,x0,5

addi s1,x0,10

addi s3,x0,4

addi s4,x0,3

beq s0,s1,TRUE # beq means branch if equal

add s2,s3,s4

j END

TRUE:

sub s2,s3,s4

END:
```

Output:

PC	Machine Code	Basic Code	Original Code	t1 (x6)
0x0	0x00500413	addi x8 x0 5	addi s0,x0,5	0
0x4	0x00A00493	addi x9 x0 10	addi s1,x0,10	0
0x8	0x00400993	addi x19 x0 4	addi s3,x0,4	5
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3	0
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal	1
				a1 (x11) 2147483612

PC	Machine Code	Basic Code	Original Code	t1 (x6)
0x0	0x00500413	addi x8 x0 5	addi s0,x0,5	0
0x4	0x00A00493	addi x9 x0 10	addi s1,x0,10	0
0x8	0x00400993	addi x19 x0 4	addi s3,x0,4	5
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3	10
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal	1
				a1 (x11) 2147483612

PC	Machine Code	Basic Code	Original Code	a5 (x15)	
0x0	0x00500413	addi x8 x0 5	addi s0,x0,5	0	
0x4	0x00A00493	addi x9 x0 10	addi s1,x0,10	0	
0x8	0x00400993	addi x19 x0 4	addi s3,x0,4	0	
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3	4	
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal	0	
PC	Machine Code	Basic Code	Original Code	a5 (x15)	
0x0	0x00500413	addi x8 x0 5	addi s0,x0,5	0	
0x4	0x00A00493	addi x9 x0 10	addi s1,x0,10	0	
0x8	0x00400993	addi x19 x0 4	addi s3,x0,4	0	
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3	4	
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal	3	
0x14	0x01498933	add x18 x19 x20	add s2,s3,s4	0	
0x8	0x00400993	addi x19 x0 4	addi s3,x0,4	0	
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3	4	
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal	3	
0x14	0x01498933	add x18 x19 x20	add s2,s3,s4	0	
0x18	0x0080006F	jal x0 8	j END	0	
0xc	0x00300A13	addi x20 x0 3	addi s4,x0,3	0	
0x10	0x00940663	beq x8 x9 12	beq s0,s1,TRUE # beq means branch if equal	0	
0x14	0x01498933	add x18 x19 x20	add s2,s3,s4	7	
0x18	0x0080006F	jal x0 8	j END	4	

Task 5: Write RISC-V assembly code to shift a number (e.g. 2) to the left twice and simulate using Venus simulator.

Description:

In this task, we performed bitwise logical left shift operations on a register using the slli instruction. We observed how shifting left by one position repeatedly doubles the register's decimal value. We executed and simulated this code in the Venus simulator and correctly interpreted the resulting bit shifts and register outputs.

Code:

```
addi s0, x0, 2
```

slli s0, s0, 1

slli s0, s0, 1

Output:

PC	Machine Code	Basic Code	Original Code	t1 (x6)	s0 (x8)
0x0	0x00200413	addi x8 x0 2	addi s0, x0, 2	0	2
0x4	0x00141413	slli x8 x8 1	slli s0, s0, 1	0	2
0x8	0x00141413	slli x8 x8 1	slli s0, s0, 1	0	1

PC	Machine Code	Basic Code	Original Code	t1 (x6)	s0 (x8)
0x0	0x00200413	addi x8 x0 2	addi s0, x0, 2	0	4
0x4	0x00141413	slli x8 x8 1	slli s0, s0, 1	0	4
0x8	0x00141413	slli x8 x8 1	slli s0, s0, 1	0	0

Task 6: Write RISC-V assembly language for given loop and simulate it using Venus.

Description:

In this task, we built a basic counter loop using the blt instruction to repeat execution until a target value was reached. The registers incremented on every pass until the condition evaluated to false. We executed and simulated this code in the Venus simulator and correctly interpreted the loop iterations and register changes.

Code:

```
addi s0,x0,0
addi s1,x0,1
Loop:
addi s0,s0,1
addi s1,s1,1
addi t0,x0,6
blt s1,t0,Loop
```

Output:

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t2 (x7)	0
0x4	0x00100493	addi x9 x0 1	addi s1,x0,1	s0 (x8)	0
0x8	0x00140413	addi x8 x8 1	addi s0,s0,1	s1 (x9)	0

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t2 (x7)	0
0x4	0x00100493	addi x9 x0 1	addi s1,x0,1	s0 (x8)	0
0x8	0x00140413	addi x8 x8 1	addi s0,s0,1	s1 (x9)	1
0xc	0x00148493	addi x9 x9 1	addi s1,s1,1	a0 (x10)	1

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t2 (x7)	0
0x4	0x00100493	addi x9 x0 1	addi s1,x0,1	s0 (x8)	1
0x8	0x00140413	addi x8 x8 1	addi s0,s0,1	s1 (x9)	1
0xc	0x00148493	addi x9 x9 1	addi s1,s1,1	a0 (x10)	1
0x10	0x00600293	addi x5 x0 6	addi t0,x0,6		

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t2 (x7)	0
0x4	0x00100493	addi x9 x0 1	addi s1,x0,1	s0 (x8)	1
0x8	0x00140413	addi x8 x8 1	addi s0,s0,1	s1 (x9)	2
0xc	0x00148493	addi x9 x9 1	addi s1,s1,1	a0 (x10)	1
0x10	0x00600293	addi x5 x0 6	addi t0,x0,6	a1 (x11)	2147483612
0x14	0xFE54CAE3	blt x9 x5 -12	blt s1,t0,Loop	a2 (x12)	0

PC	Machine Code	Basic Code	Original Code	ra (x1)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	sp (x2)	2147483612
0x4	0x00100493	addi x9 x0 1	addi s1,x0,1	gp (x3)	268435456
0x8	0x00140413	addi x8 x8 1	addi s0,s0,1	tp (x4)	0
0xc	0x00148493	addi x9 x9 1	addi s1,s1,1	t0 (x5)	6
0x10	0x00600293	addi x5 x0 6	addi t0,x0,6	t1 (x6)	0
0x14	0xFE54CAE3	blt x9 x5 -12	blt s1,t0,Loop		

Task 7: Write RISC-V assembly language for given loop and simulate it using Venus.

Description:

In this task, we calculated array memory addresses using byte offsets and stored calculated values into data memory using sw. A loop executed five times to sequentially write the values into consecutive memory slots. We executed and simulated this code in the Venus simulator and correctly interpreted the memory updates and address calculations.

Code:

```
.data
```

```
num: .word 0,0,0,0,0,0,0,0,0,0
```

```
.text
```

```
main:
```

```
    addi s0,x0,0
```

```
    la s1,num
```

```
Loop:
```

```
    addi t0,s0,5
```

```
    slli t1,s0,2
```

```
    add t2,s1,t1
```

```
    sw t0,0(t2)
```

```
    addi s0,s0,1
```

```
    addi t3,x0,5
```

```
    blt s0,t3,Loop
```

Output:

PC	Machine Code	Basic Code	Original Code	tp (x4)
0x0	0x00000413	addi x8 x0 0	addi s0,x0,0	
0x4	0x10000497	auipc x9 65536	la s1,num	
0x8	0xFFC48493	addi x9 x9 -4	la s1,num	

PC	Machine Code	Basic Code	Original Code	t1 (x6)
0x0	0x00000413	addi x8 x0 0	addi s0,x0,0	
0x4	0x10000497	auipc x9 65536	la s1,num	
0x8	0xFFC48493	addi x9 x9 -4	la s1,num	

PC	Machine Code	Basic Code	Original Code	t1 (x6)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t2 (x7)	0
0x4	0x1000497	auipc x9 65536	la s1,num	s0 (x8)	0
0x8	0xFFC48493	addi x9 x9 -4	la s1,num	s1 (x9)	268435460
0xc	0x00540293	addi x5 x8 5	addi t0,s0,5	a0 (x10)	1
0x10	0x00241313	slli x6 x8 2	slli t1,s0,2	t1 (x6)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t2 (x7)	0
0x4	0x1000497	auipc x9 65536	la s1,num	s0 (x8)	0
0x8	0xFFC48493	addi x9 x9 -4	la s1,num	s1 (x9)	268435456
0xc	0x00540293	addi x5 x8 5	addi t0,s0,5	a0 (x10)	1
0x10	0x00241313	slli x6 x8 2	slli t1,s0,2	BP (x3)	268435456
0x8	0xFFC48493	addi x9 x9 -4	la s1,num	tp (x4)	0
0xc	0x00540293	addi x5 x8 5	addi t0,s0,5	t0 (x5)	5
0x10	0x00241313	slli x6 x8 2	slli t1,s0,2	tp (x4)	0
0xc	0x00540293	addi x5 x8 5	addi t0,s0,5	t0 (x5)	5
0x10	0x00241313	slli x6 x8 2	slli t1,s0,2	t1 (x6)	0
0x14	0x006483B3	add x7 x9 x6	add t2,s1,t1	t0 (x5)	5
0x10	0x00241313	slli x6 x8 2	slli t1,s0,2	t1 (x6)	0
0x14	0x006483B3	add x7 x9 x6	add t2,s1,t1	t2 (x7)	268435456
0x18	0x0053A023	sw x5 0(x7)	sw t0,0(t2)	t1 (x6)	0
0x14	0x006483B3	add x7 x9 x6	add t2,s1,t1	t2 (x7)	268435456
0x18	0x0053A023	sw x5 0(x7)	sw t0,0(t2)	s0 (x8)	0
0x1c	0x00140413	addi x8 x8 1	addi s0,s0,1	t1 (x6)	0
0x1c	0x00140413	addi x8 x8 1	addi s0,s0,1	t2 (x7)	268435456
0x20	0x00500E13	addi x28 x0 5	addi t3,x0,5	s0 (x8)	1
0x24	0xFFC444E3	blt x8 x28 -24	blt s0,t3,loop	(x27)	
0x1c	0x00140413	addi x8 x8 1	addi s0,s0,1	t3 (x28)	5
0x20	0x00500E13	addi x28 x0 5	addi t3,x0,5	t4 (x29)	0
0x24	0xFFC444E3	blt x8 x28 -24	blt s0,t3,loop		

Task 8: Write RISC-V assembly code for nested loops and simulate it using Venus.

Description:

In this task, we structured a nested loop where an inner loop ran multiple times within each cycle of an outer loop. The inner addition executed six times total before both loop conditions were satisfied. We executed and simulated this code in the Venus simulator and correctly interpreted the step-by-step register updates across all iterations.

Code:

addi s0,x0,0

addi s1,x0,0

Outer:

addi s2,x0,0

Inner:

add s0,s1,s2

addi s2,s2,1

addi t0,x0,2

blt s2,t0,Inner

addi s1,s1,1

addi t1,x0,3

blt s1,t1,Outer

Output:

PC	Machine Code	Basic Code	Original Code	s6 (x22)	e
0x0	0x00000413	addi x8 x0 0	addi s0,x0,0	s7 (x23)	e
0x4	0x00000493	addi x9 x0 0	addi s1,x0,0	s8 (x24)	e
0x8	0x00000913	addi x18 x0 0	addi s2,x0,0	s9 (x25)	e
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	s10 (x26)	e
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	s11 (x27)	e
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	t3 (x28)	e
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner		

PC	Machine Code	Basic Code	Original Code	t0 (x5)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t1 (x6)	0
0x4	0x0000493	addi x9 x0 0	addi s1,x0,0	t2 (x7)	0
0x8	0x0000913	addi x18 x0 0	addi s2,x0,0	s0 (x8)	0
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	s1 (x9)	0
0x0	0x0000413	addi x8 x0 0	addi s0,x0,0	t1 (x6)	0
0x4	0x0000493	addi x9 x0 0	addi s1,x0,0	t2 (x7)	0
0x8	0x0000913	addi x18 x0 0	addi s2,x0,0	s0 (x8)	0
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	s1 (x9)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	a0 (x10)	1
0x4	0x0000493	addi x9 x0 0	addi s1,x0,0	a7 (x17)	0
0x8	0x0000913	addi x18 x0 0	addi s2,x0,0	s2 (x18)	0
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	s3 (x19)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	s4 (x20)	0
0x8	0x0000913	addi x18 x0 0	addi s2,x0,0	t1 (x6)	0
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	t2 (x7)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	s0 (x8)	0
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	s1 (x9)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	a6 (x16)	0
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	a7 (x17)	0
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner	s2 (x18)	1
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	tp (x4)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	t0 (x5)	2
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	t1 (x6)	0
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner	t2 (x7)	0
0x8	0x0000913	addi x18 x0 0	addi s2,x0,0	gp (x3)	268435456
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	tp (x4)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	t0 (x5)	2
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	t2 (x7)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	s0 (x8)	1
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	s1 (x9)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	s2 (x18)	2
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	s3 (x19)	0
0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	tp (x4)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	t0 (x5)	2
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	t1 (x6)	0
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner	t2 (x7)	0

0xc	0x01248433	add x8 x9 x18	add s0,s1,s2	tp (x4)	0
0x10	0x00190913	addi x18 x18 1	addi s2,s2,1	t0 (x5)	2
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	t1 (x6)	8
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner	t2 (x7)	8
0x1c	0x00148493	addi x9 x9 1	addi s1,s1,1	s0 (x8)	1
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner	t2 (x7)	8
0x1c	0x00148493	addi x9 x9 1	addi s1,s1,1	s0 (x8)	1
0x20	0x00300313	addi x6 x0 3	addi t1,x0,3	s1 (x9)	1
0x14	0x00200293	addi x5 x0 2	addi t0,x0,2	t0 (x5)	2
0x18	0xFE594AE3	blt x18 x5 -12	blt s2,t0,Inner	t1 (x6)	3
0x1c	0x00148493	addi x9 x9 1	addi s1,s1,1	t2 (x7)	8
0x20	0x00300313	addi x6 x0 3	addi t1,x0,3	s0 (x8)	1
0x24	0xFE64C2E3	blt x9 x6 -28	blt s1,t1,Outer		